SORTING ALGORITHMS

- There are many ways a programmer can choose to sort a list of numbers, from simple methods such as C++'s std::sort, bubble sorts and insertion sorts - to more complex methods such as Radix and heap sorts.
  - There are also methods designed to look cool when visualised such as a shaker sort but are also very inefficient.

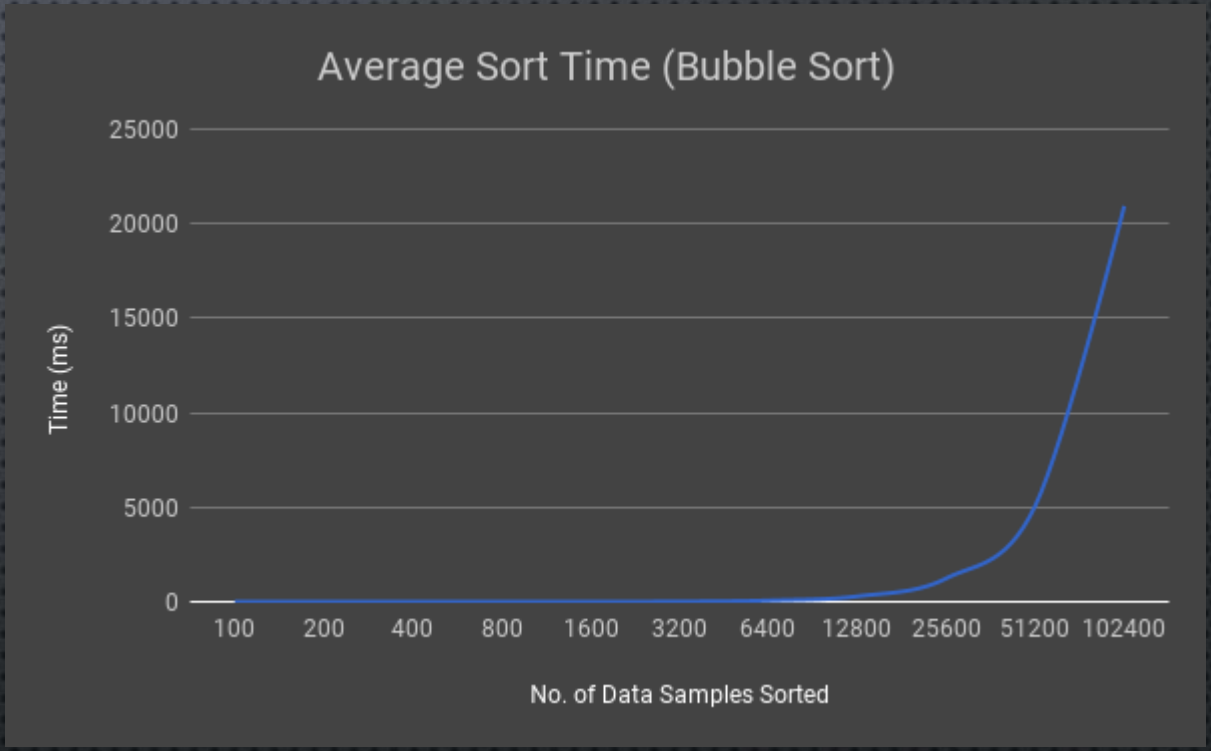
DATA STRUCTURE

- VECTORS WERE CHOSEN AS AN APPROPRIATE DATA STRUCTURE AS THEY ALLOW FOR DYNAMIC RESIZING
  - THIS MEANS THAT THE PROGRAM CAN EASILY BE ADJUSTED TO GENERATE MORE DATA SAMPLES, OTHER METHODS SUCH AS ARRAYS WOULD REQUIRE EXTRA LINES OF CODE.
  - THEY ARE ALSO SLIGHTLY SLOWER TO ACCESS HOWEVER AS THE POINTERS TO THE HEAP HAVE TO BE CHASED TO GET ARRAYED DATA.

Information reference - https://stackoverflow.com/questions/15079057/arrays-vs-vectors-introductory-similarities-and-differences

BUBBLE SORTS

- A BUBBLE SORT WORKS BUY SWAPPING AN INTEGER WITH THE NEXT VALUE IN THE VECTOR IF IT IS LARGER, ALLOWING THE LARGER NUMBERS TO BUBBLE UP TO THE TOP.
  - THIS WORKS WELL IN THEORY FOR SMALL COLLECTIONS OF LARGE NUMBERS, HOWEVER AS DATA SIZE INCREASES THE TIME TAKEN TO SORT THE DATA INCREASES EXPONENTIALLY (AS SEEN IN GRAPH 1.)
  - TIME COMPLEXITY OF O(N) IN BEST CASE, AND O(N^2) ON AVERAGE, WHERE "N" IS THE NUMBER OF VALUES.



Graph 1 – Average Sort Time for a Bubble sort (increasing data size but not range)

PSEUDOCODE

```
for all elements of list (counter < size of vector)
        if swap = true
                if list[counter] > list[counter+1]
                        swap(list[counter], list[counter+1])
                        swap = true
                else swap = false (vector is sorted)
                end if
   end for
```

RADIX SORT

- Radix sorts work by sorting a list of numbers one decimal place at a time, then incrementing to the next decimal place and doing another pass.
  - This allows them to be extremely efficient.
  - Time complexity of O(nk) where "k" is the number of decimal places in the largest number and "n" is the number of values.

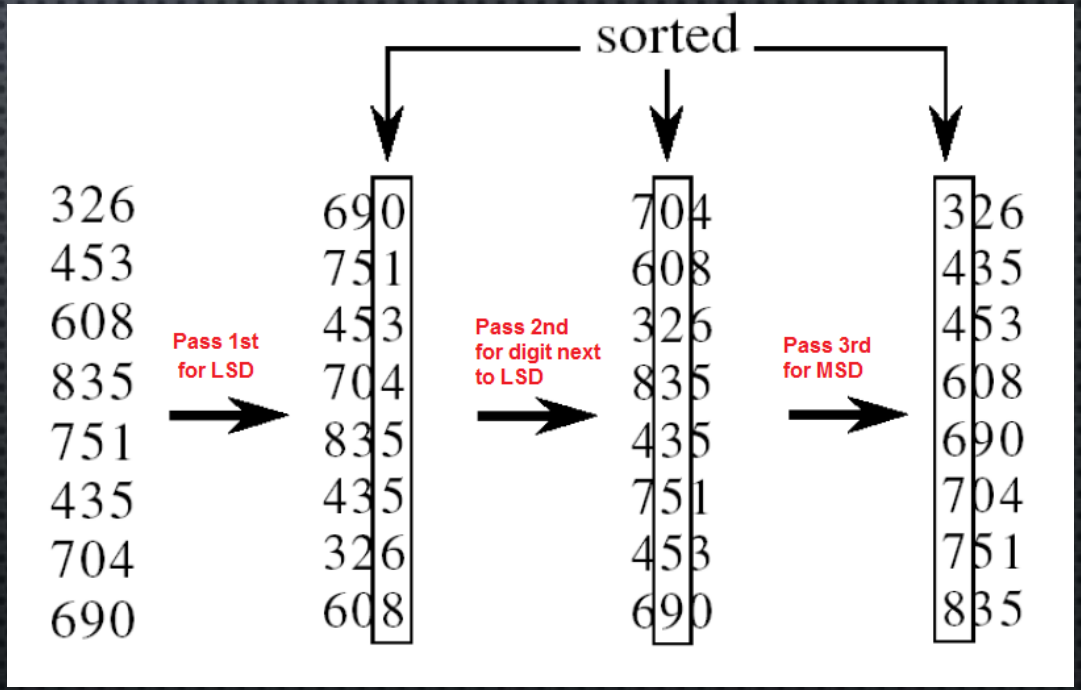


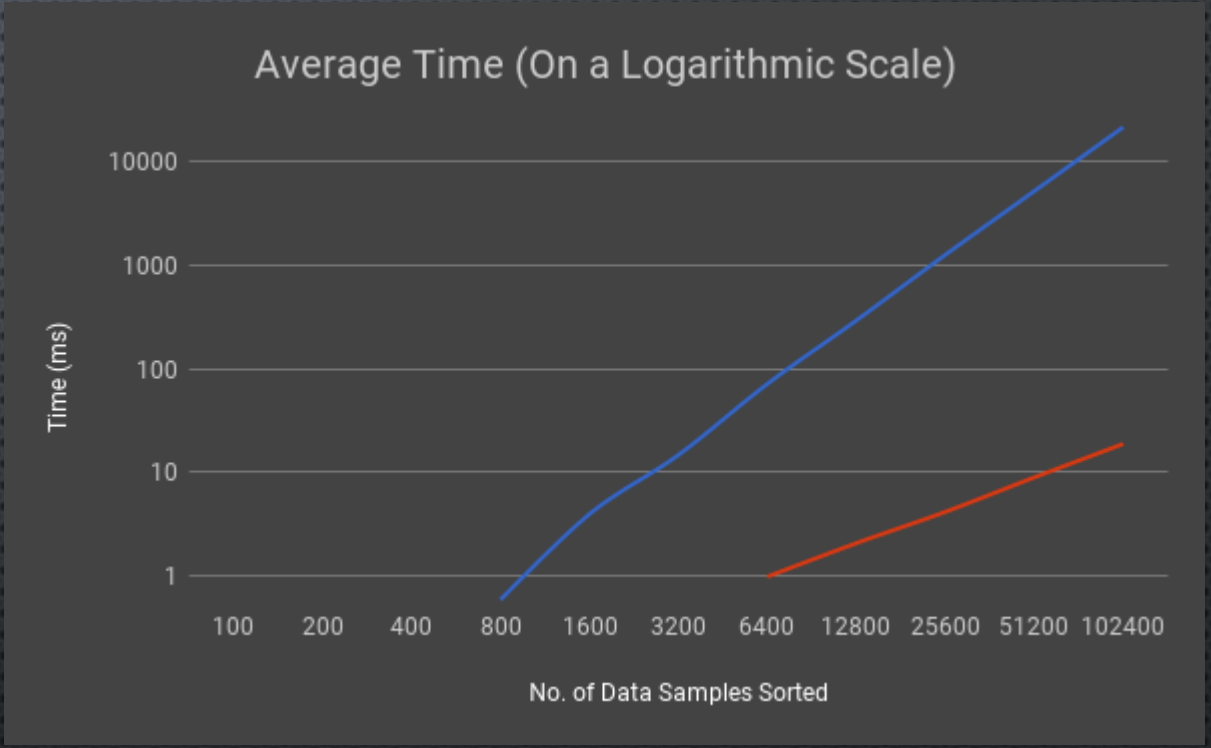
Image Source - http://www.codingeek.com/wp-content/uploads/2017/02/radix.png

PSEUDOCODE

```
for all elements of list (counter < size of vector)

     sort by least significant digit

     sort by tens

     sort by hundreds

     sort by thousands

     sort by tens of thousands

end for
```

COMPARISON

- Unsurprisingly, radix sort was by far more efficient when compared to a bubble sort.
  - The graph shows a logarithmic scale of comparison as a standard graph made the radix sort line almost flat.
  - The graph shows that on average, the bubble sort was roughly 1000x slower than the radix sort.



QUESTIONS?